SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	Abinaya Sri J	**Reg. No.:**	192524429
Date:	24/10/26	**Duration:**	60 minutes

Code of Conduct

I Abinaya Sri J (192524429) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the Student:

1. Study and Analyze the Scenario

1.1 Introduction

Healthcare facilities require an efficient way to coordinate patients, doctors, appointments, and consultation records. When these activities depend on manual registers, telephone calls, or disconnected software, information may become outdated, appointment conflicts may occur, and administrative tasks may take more time.

The **Smart Healthcare Appointment and Patient Management System** is a centralized web-based application designed to improve appointment scheduling and patient information management. It provides separate access for patients, doctors, receptionists, and hospital administrators. The system aims to improve operational efficiency, reduce scheduling errors, protect confidential information, and provide a convenient healthcare service experience.

1.2 Business Domain

The proposed system belongs to the **Healthcare Information Management and Hospital Administration** domain.

It focuses on the following business activities:

- Patient registration and profile management.
- Doctor search and availability management.
- Appointment booking, cancellation, and rescheduling.
- Doctor schedules and consultation record management.
- Hospital administration and appointment reporting.
- Secure access to patient and appointment information.

1.3 Problem Description

In a traditional healthcare appointment process, patients may need to contact the hospital or visit the reception desk to check doctor availability and schedule consultations. Receptionists maintain appointment information, while doctors manage their schedules and consultation records separately. This can result in duplicate appointments, long waiting times, delayed information sharing, and difficulties retrieving patient records.

The proposed system addresses these problems by providing a centralized platform where authorized users can access updated information and complete their respective activities. Appointment availability is validated before a booking is confirmed, and role-based permissions ensure that sensitive information is accessible only to authorized users.

1.4 Objectives of the System

The main objectives are:

1. To develop a centralized platform for managing patient appointments and healthcare administrative activities.
2. To enable patients to search for doctors and book available appointment slots conveniently.
3. To reduce appointment conflicts through availability validation and consistent booking operations.
4. To help doctors manage their schedules and access authorized consultation information.
5. To enable receptionists and administrators to manage appointments, users, and reports efficiently.
6. To protect patient information through authentication, authorization, and secure data handling.
7. To improve the accessibility, reliability, usability, and maintainability of healthcare management operations.

1.5 Scope of the System

In scope

- Patient registration, login, and profile management.
- Doctor profiles, specializations, and availability.
- Appointment booking, cancellation, rescheduling, and status tracking.
- Doctor schedules and consultation records.
- Receptionist-assisted appointment management.
- Administrative management of users, doctors, and appointments.
- Appointment notifications and basic operational reports.
- Role-based access control, activity logging, and data backup.

Out of scope for the initial version

- Automated medical diagnosis and treatment recommendations.
- Emergency ambulance dispatch and emergency-care management.
- Direct integration with all existing hospital information systems.
- Advanced insurance claim processing.
- Full electronic health record interoperability across multiple hospitals.

These exclusions keep the initial project focused on appointment and patient-management workflows while allowing future expansion.

1.6 Stakeholders and Their Requirements

Stakeholder	Responsibilities	Main requirements
Patients	Search for doctors and manage appointments	Convenient booking, accurate availability, appointment status, privacy
Doctors	Manage schedules and consultation records	Updated daily schedules and authorized access to patient information
Receptionists	Assist with registration and appointment scheduling	Quick patient lookup and efficient booking management
Hospital administrators	Manage users, doctors, appointments, and reports	Centralized control and accurate operational information
System administrators	Maintain application infrastructure and security	Monitoring, backup, recovery, and access management
Hospital management	Review service performance and operational trends	Reliable reports and appointment statistics
Development and testing team	Build, verify, and maintain the application	Clear requirements, testable features, and maintainable architecture

1.7 Existing Challenges

The major challenges in the existing process include:

- **Appointment conflicts:** Manual updates can cause overlapping bookings.
- **Limited availability information:** Patients may not know whether a doctor has an available slot.
- **Information fragmentation:** Patient details and appointment records may be stored in different places.
- **Administrative workload:** Receptionists may spend significant time answering booking enquiries and updating records.
- **Communication delays:** Patients may not receive timely appointment confirmations or cancellation updates.
- **Data confidentiality:** Poorly controlled access can expose sensitive patient information.
- **Reporting difficulties:** Manual preparation of appointment statistics can be time-consuming and error-prone.

2. Functional and Non-Functional Requirements

2.1 Functional Requirements

Functional requirements specify the operations that the system must perform. Each requirement is assigned a unique identifier to support design, implementation, and testing.

ID	Functional requirement	Priority
FR-01	The system shall allow patients to create an account using the required registration details.	High
FR-02	The system shall authenticate registered users and provide access according to their assigned roles.	High
FR-03	The system shall allow authorized users to view and update permitted profile information.	High
FR-04	The system shall display active doctor profiles, specializations, and relevant details.	High
FR-05	The system shall display available appointment dates and time slots for a selected doctor.	High
FR-06	The system shall allow patients to book an available appointment slot.	High
FR-07	The system shall validate slot availability during booking and prevent conflicting confirmed appointments for the same doctor and time.	High
FR-08	The system shall allow patients to cancel or reschedule appointments according to the configured appointment policy.	High
FR-09	The system shall display appointment confirmation, status, and booking history to authorized users.	High
FR-10	The system shall allow doctors to create and update their availability schedules.	High
FR-11	The system shall allow doctors to view their daily and upcoming appointments.	High
FR-12	The system shall allow authorized doctors to record and retrieve consultation notes for relevant appointments.	High
FR-13	The system shall allow receptionists to register patients when permitted and assist with appointment booking and rescheduling.	Medium
FR-14	The system shall allow administrators to add, update, deactivate, and manage doctor and user accounts.	High
FR-15	The system shall allow administrators to monitor appointment records and generate date-based operational reports.	Medium
FR-16	The system shall send booking confirmations and appointment-status notifications through a configured notification service.	Medium
FR-17	The system shall record relevant administrative and security-sensitive activities in an activity log.	Medium
FR-18	The system shall support authorized backup and recovery procedures for stored data.	High

2.2 Non-Functional Requirements

Non-functional requirements specify the expected quality, performance, and operational characteristics of the system.

Category	Requirement
Performance	Under the agreed normal test load, at least 95% of common page requests and doctor searches should complete within 3 seconds, excluding external service delays.

Security	The system shall use secure authentication, role-based access control, HTTPS, and protected password storage.
Reliability	The system shall preserve confirmed appointment records and handle booking failures without creating duplicate confirmed bookings.
Usability	The interface shall use clear labels, understandable navigation, and consistent layouts across user roles.
Scalability	The architecture shall support increasing numbers of patients, doctors, appointments, and reports without requiring a complete redesign.
Maintainability	The application shall separate user management, appointment management, consultation management, and reporting into identifiable components.
Availability	The proposed initial target shall be 99.5% monthly availability, excluding agreed maintenance, subject to stakeholder approval.
Data integrity	The system shall validate required fields, enforce unique identifiers, and maintain valid relationships between database records.
Compatibility	The web application shall support current versions of commonly used browsers and responsive desktop and mobile layouts.
Backup and recovery	The system shall perform scheduled backups according to an agreed recovery policy and periodically test restoration.
Privacy	Patient and consultation information shall only be disclosed to authorized users for permitted purposes.
Accessibility	The interface should provide readable text, keyboard-accessible controls, and meaningful labels for important form fields.

3. Actors and Use Case Modelling

3.1 Primary Actors

1. Patient

The patient is the main user of the appointment-booking service. The patient can register, log in, manage permitted profile information, search for doctors, view available slots, book appointments, cancel or reschedule bookings, and view appointment history.

2. Doctor

The doctor manages availability, views assigned appointments, updates appointment status, and records consultation notes for relevant patients.

3. Receptionist

The receptionist assists patients with registration and appointment scheduling, searches for patient records within permitted access, and helps manage booking changes.

4. Hospital Administrator

The administrator manages doctor and user accounts, monitors appointment information, maintains schedules within assigned permissions, and generates operational reports.

3.2 Secondary Actors

1. Notification Service

An external email, SMS, or notification provider sends appointment confirmations, cancellation messages, and reminders when the relevant service is configured.

2. System Administrator

The system administrator maintains technical configuration, application availability, access policies, system monitoring, and backup and recovery operations.

3.3 Major Use Cases by Actor

Actor	Use cases
Patient	Register, log in, manage profile, search doctors, view availability, book appointment, cancel/reschedule appointment, view appointment history
Doctor	Log in, manage availability, view schedule, update appointment status, record consultation notes
Receptionist	Register patient where permitted, search patient, book appointment, reschedule appointment, view appointment status
Hospital Administrator	Manage users, manage doctors, manage schedules, monitor appointments, generate reports
Notification Service	Deliver booking confirmations, cancellation notifications, and appointment reminders
System Administrator	Manage system configuration, review technical logs, manage backups, support recovery

3.4 UML Use Case Diagram

The following PlantUML code can be rendered into a UML diagram and inserted as an image into the Word document. It shows the system boundary, actors, use cases, and the relationship between appointment booking and notifications.

```
@startuml

left to right direction

skinparam packageStyle rectangle

skinparam shadowing false

actor Patient

actor Doctor

actor Receptionist

actor "Hospital Administrator" as Admin

actor "Notification Service" as Notify

actor "System Administrator" as SysAdmin
```

```
rectangle "Smart Healthcare Appointment and Patient Management System" {
  usecase "Register / Log In" as UC1
  usecase "Manage Profile" as UC2
  usecase "Search Doctors" as UC3
  usecase "View Available Slots" as UC4
  usecase "Book Appointment" as UC5
  usecase "Validate Slot Availability" as UC6
  usecase "Cancel / Reschedule Appointment" as UC7
  usecase "View Appointment History" as UC8
  usecase "Manage Availability" as UC9
  usecase "View Daily Schedule" as UC10
  usecase "Update Appointment Status" as UC11
  usecase "Record Consultation Notes" as UC12
  usecase "Assist with Booking" as UC13
  usecase "Manage Users and Doctors" as UC14
  usecase "Manage Schedules" as UC15
  usecase "Generate Reports" as UC16
  usecase "Send Appointment Notification" as UC17
  usecase "Manage System and Backups" as UC18
}
Patient --> UC1
Patient --> UC2
Patient --> UC3
Patient --> UC4
Patient --> UC5
Patient --> UC7
```

Patient --> UC8

Doctor --> UC1

Doctor --> UC9

Doctor --> UC10

Doctor --> UC11

Doctor --> UC12

Receptionist --> UC13

Receptionist --> UC7

Receptionist --> UC8

Admin --> UC14

Admin --> UC15

Admin --> UC16

Notify --> UC17

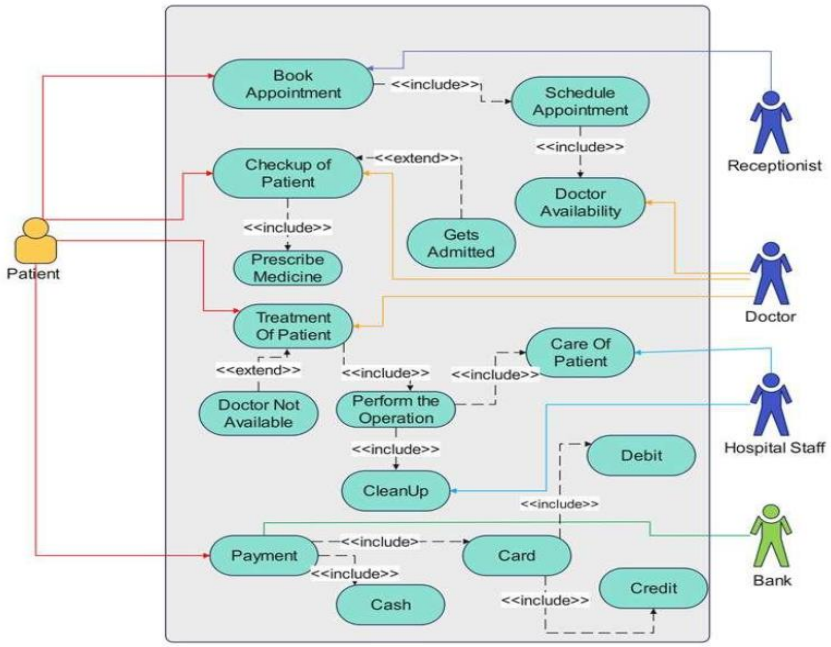
SysAdmin --> UC18

UC5 ..> UC6 : <<include>>

UC5 ..> UC17 : <<include>>

UC7 ..> UC17 : <<include>>

@enduml



Explanation: The system boundary contains the use cases provided by the application. Actors remain outside the boundary and interact with the relevant functions. The include relationship indicates that booking requires slot validation and that the system initiates the notification workflow for relevant booking events. Actual delivery depends on the configured notification service.

3.5 Detailed Use Case Descriptions

Use Case 1: Book Appointment

Item	Description
Use Case ID	UC-01
Primary actor	Patient
Supporting actor	Notification Service
Preconditions	The patient is authenticated, and the selected doctor has an available slot.
Trigger	The patient selects the option to book an appointment.

Main flow

1. The patient searches for a doctor by name or specialization.
2. The system displays the doctor's profile and available slots.
3. The patient selects a suitable date and time.
4. The system validates the selected slot.
5. The patient confirms the booking.
6. The system creates the appointment using a unique appointment ID and saves it with a confirmed status.

7. The system updates slot availability and displays the booking confirmation.
8. The system requests a confirmation notification through the configured notification service.

Alternative flow

- If the selected slot is no longer available, the system displays a message and requests another slot.
- If required information is missing, the system identifies the invalid fields and requests correction.
- If saving fails, the system does not display a successful confirmation and informs the patient that the booking could not be completed.

Postconditions: A successful booking is stored consistently, and the slot cannot be confirmed for another overlapping appointment with the same doctor.

Use Case 2: Manage Doctor Availability

Item	Description
Use Case ID	UC-02
Primary actor	Doctor
Preconditions	The doctor is authenticated and authorized to manage their schedule.
Trigger	The doctor opens the availability-management page.

Main flow

1. The doctor views the current schedule.
2. The doctor selects working dates and time slots.
3. The system validates the entered dates and times.
4. The system checks for conflicts with existing confirmed appointments.
5. The system saves valid schedule changes.
6. The updated availability becomes visible to authorized users.

Alternative flow: If a proposed change conflicts with a confirmed appointment, the system warns the doctor and requires the conflict to be resolved according to hospital policy.

Postconditions: The updated schedule is stored, and existing confirmed appointments are not silently removed.

Use Case 3: Generate Appointment Report

Item	Description
Use Case ID	UC-03
Primary actor	Hospital Administrator
Preconditions	The administrator is authenticated and has reporting permission.

Trigger	The administrator opens the reporting section.

Main flow

1. The administrator selects a report type and date range.
2. The system validates the selected filters.
3. The system retrieves the relevant appointment information.
4. The system calculates and displays the requested summary.
5. The administrator views or exports the report if the export function is enabled.

Alternative flow: If no matching records exist, the system displays an appropriate message rather than an empty or misleading report.

Postconditions: The report is displayed to the authorized administrator without exposing unnecessary patient-sensitive information.

4. Software Requirement Specification (SRS)

4.1 Purpose

This Software Requirement Specification defines the expected behaviour, quality attributes, interfaces, and data requirements of the Smart Healthcare Appointment and Patient Management System. It provides a common reference for stakeholders, developers, testers, and system administrators throughout development and maintenance.

4.2 Intended Users

The intended users are patients, doctors, receptionists, hospital administrators, and system administrators. The development and testing team will use the SRS to design the application, implement its features, and verify that the delivered system meets the agreed requirements.

4.3 Overall System Description

The system is a centralized web application with role-specific interfaces. Users access the application through a supported web browser. The application processes requests, validates appointment rules, communicates with the database, and requests notifications when required.

The proposed architecture separates the presentation layer, application logic, and data storage. This separation improves maintainability, testing, security enforcement, and future scalability.

4.4 System Architecture Diagram

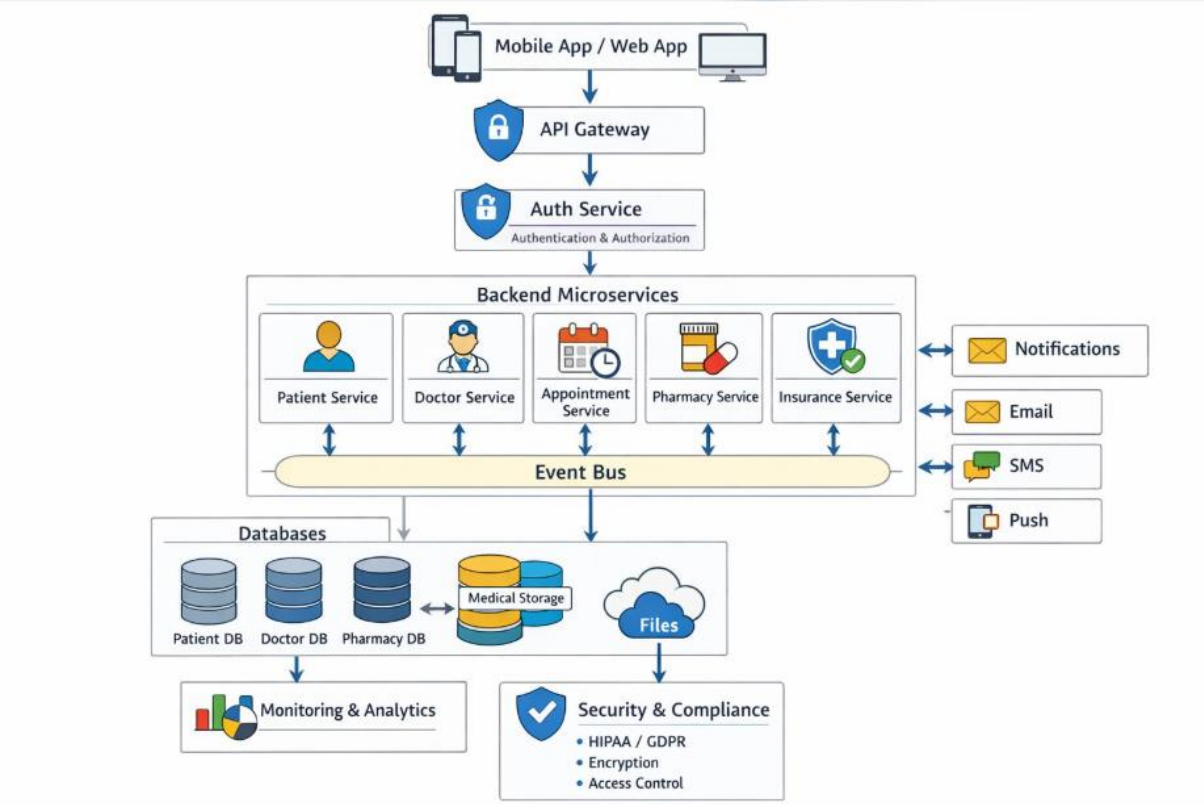


Figure 2: System Architecture of the Smart Healthcare Appointment and Patient Management System.

Explanation: The system architecture illustrates the integration of patient, doctor, appointment, pharmacy, and insurance services through an API Gateway and secure authentication. It also includes databases, notification services, monitoring, and security mechanisms to ensure efficient healthcare management and secure patient data handling.

4.5 Functional Requirements

The detailed functional requirements are listed in Section 2.1 and identified by FR-01 to FR-18. Each requirement shall be mapped to a design component and one or more test cases before implementation is accepted.

4.6 Non-Functional Requirements

The quality attributes and proposed measurable targets are specified in Section 2.2. They will be verified using performance tests, security tests, usability evaluation, recovery tests, and code-level maintainability reviews.

4.7 Data Requirements

The database should contain the following entities.

Entity	Main attributes	Purpose
Patient	PatientID (PK), Name, Contact, Email, AccountID (FK)	Stores patient identity and contact information

Doctor	DoctorID (PK), Name, Specialization, Contact, AccountID (FK), Status	Stores doctor profile information
Availability	AvailabilityID (PK), DoctorID (FK), Date, StartTime, EndTime, Status	Stores doctor schedules and time slots
Appointment	AppointmentID (PK), PatientID (FK), DoctorID (FK), AvailabilityID (FK), Status, CreatedAt	Stores appointment bookings
Consultation	ConsultationID (PK), AppointmentID (FK), DoctorID (FK), ConsultationDate, Notes	Stores authorized consultation information
UserAccount	AccountID (PK), LoginIdentifier, PasswordHash, Role, AccountStatus	Stores authentication references and user roles
Notification	NotificationID (PK), AppointmentID (FK), Recipient, Type, DeliveryStatus, SentAt	Tracks notification requests and delivery status
ActivityLog	LogID (PK), AccountID (FK), Action, Timestamp	Records relevant user and administrative activities

Data integrity rules

- Each entity shall have a unique primary key.
- Foreign keys shall reference valid records.
- An appointment shall reference an existing patient and doctor.
- The system shall enforce booking conflict checks at the database transaction level or through an equivalent concurrency-safe mechanism.
- Consultation notes shall be accessible only to authorized users.
- Passwords shall be stored as secure password hashes, not plain text.
- Sensitive data shall not be included unnecessarily in application logs.
- Data retention and deletion shall follow approved organizational policy and applicable requirements.

4.8 Entity Relationship Diagram (ERD)

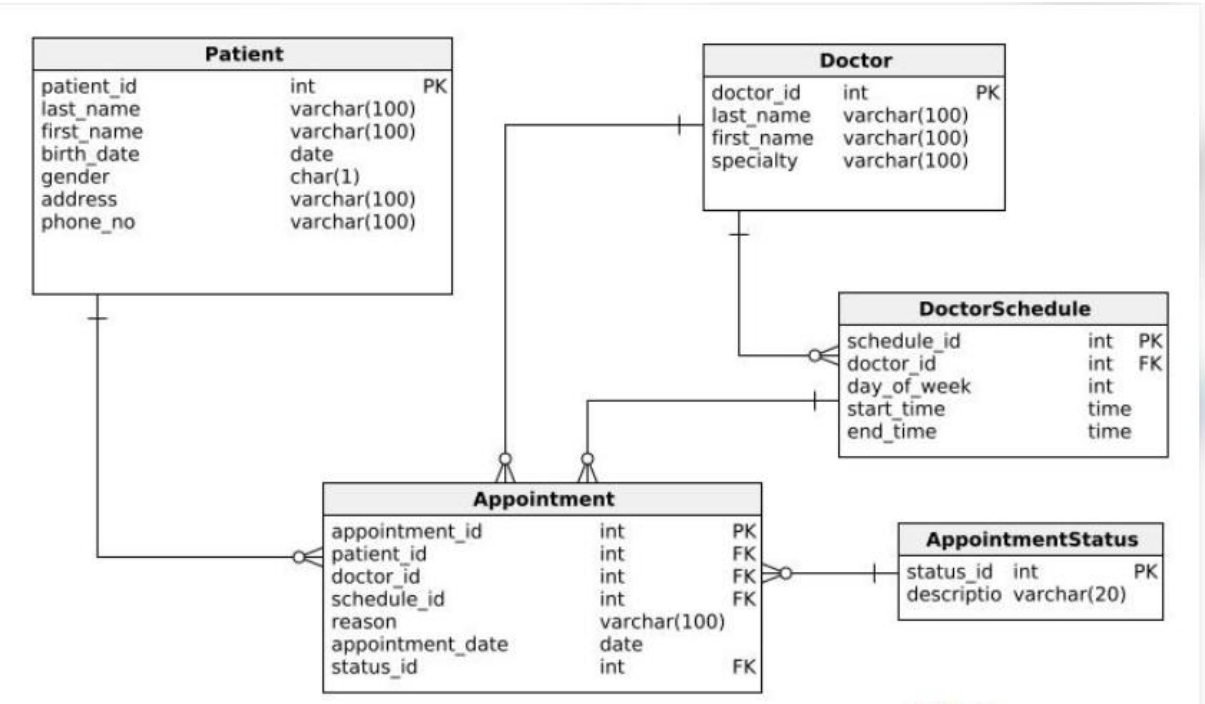


Figure 3: Proposed Database Relationships

Explanation: One patient may have many appointments, and one doctor may manage many appointments. Each appointment references the selected availability record. An appointment may have a consultation record when a consultation takes place. The exact relationship constraints should be enforced in the database design.

4.9 External Interface Requirements

User interface

- The system shall provide separate role-based dashboards.
- Forms shall validate required fields and display understandable error messages.
- Appointment status shall be displayed consistently across relevant pages.
- The interface shall support desktop and mobile screen sizes.

Software interface

- A web application server shall process requests and apply business rules.
- A relational database shall store user, doctor, appointment, and consultation data.
- A notification provider may be integrated for email or SMS delivery.
- Database access shall be restricted to authorized application components.

Communication interface

- Browser-to-server communication shall use HTTPS.

- External integrations shall use their supported secure communication protocols.
- Authentication secrets and API credentials shall not be exposed in client-side code.

Hardware interface

- Users require an internet-connected desktop, laptop, tablet, or smartphone with a supported browser.
- No specialized hardware is required for the initial version.

4.10 System Features

Feature 1: User and Access Management

Supports registration, authentication, profile management, and role-based permissions.

Feature 2: Doctor Search and Availability

Displays doctor profiles, specializations, schedules, and available time slots.

Feature 3: Appointment Management

Supports booking, cancellation, rescheduling, status tracking, and appointment history while preventing conflicting confirmed bookings.

Feature 4: Consultation Management

Allows authorized doctors to access relevant appointment information and maintain consultation notes.

Feature 5: Administration and Reporting

Provides authorized administrators with tools to manage users, doctors, schedules, appointments, and operational reports.

Feature 6: Notifications and System Monitoring

Supports configured appointment notifications, relevant activity logging, and technical monitoring.

5. Assumptions and Constraints

5.1 Assumptions

1. Patients, doctors, and staff have access to a supported device and internet connection.
2. Users provide accurate registration and contact details.
3. Doctors or authorized staff maintain up-to-date availability schedules.
4. Hospital management defines working hours, appointment duration, and cancellation policies.

5. The notification service is configured before notifications are enabled.
6. Consultation records are entered and reviewed by authorized medical personnel.
7. Hospital management defines user roles and permitted access to patient information.
8. The hospital provides the necessary staff cooperation, operational rules, and training.

5.2 Constraints

Constraint	Description	Possible impact
Technical	The application depends on network access, database availability, and supported browser technology.	Service interruptions may prevent booking or record retrieval.
Operational	Staff must maintain accurate doctor schedules and follow defined procedures.	Incorrect schedules may lead to unsuitable availability information.
Economic	Development, hosting, maintenance, backups, and notification services must remain within the approved budget.	Some advanced features may need to be postponed.
Legal and privacy	Patient information must be handled according to applicable privacy, security, and healthcare requirements.	Additional controls, consent procedures, or compliance reviews may be necessary.
Time-related	The first release must be completed within the agreed project schedule.	Advanced integrations may be moved to later releases.
Integration	Existing hospital systems may use different formats and interfaces.	Integration may require additional development, testing, and approvals.
Data quality	Existing patient records may be incomplete or duplicated.	Data cleaning and validation may be needed before migration.

5.3 Cross-Disciplinary Considerations

The project involves software engineering, healthcare operations, database design, cybersecurity, user-interface design, and data privacy. Input from hospital staff helps define realistic scheduling workflows, while security and database expertise helps protect information and maintain reliable records. This collaboration supports the CO2 requirement to consider cross-disciplinary skills.

6. Validate Requirement Completeness and Consistency

6.1 Stakeholder Requirement Traceability

The following table maps key stakeholder needs to the specified requirements.

Stakeholder need	Requirement IDs	Validation method
Patients need secure access	FR-01, FR-02, FR-03	Registration, login, and permission tests
Patients need accurate availability and booking	FR-04, FR-05, FR-06, FR-07	Availability and concurrent-booking tests
Patients need to manage appointments	FR-08, FR-09, FR-16	Cancellation, rescheduling, status, and notification tests

Doctors need schedule management	FR-10, FR-11	Schedule-update and daily-view tests
Doctors need consultation records	FR-12	Authorization and consultation-record tests
Receptionists need booking support	FR-13	Receptionist workflow and permission tests
Administrators need management and reporting	FR-14, FR-15	Administrative and report-generation tests
Technical staff need reliable recovery	FR-17, FR-18	Activity-log, backup, and restoration tests
All users need usable and secure service	Non-functional requirements	Usability, security, and performance testing

6.2 Requirement Validation Checklist

Validation aspect	Check performed	Result
Completeness	Registration, booking, cancellation, schedule management, consultations, reporting, and security are covered.	Covered in the specification
Consistency	Appointment and availability rules are defined to prevent conflicting confirmed bookings.	Consistent, subject to implementation tests
Unambiguity	Functional requirements use the term “shall,” and quality targets are measurable where practical.	Improved clarity
Redundancy	Requirements are identified by unique IDs and grouped by function.	Major duplication avoided
Feasibility	The initial scope uses standard web, application, and database technologies.	Feasible, subject to resources and schedule
Testability	Functional requirements are mapped to validation methods.	Testable
Traceability	Stakeholder needs are linked to requirement IDs and proposed tests.	Traceability established
Security	Role permissions and access to consultation records are specified.	Security requirements identified
Missing requirements	Cancellation rules, recovery targets, record retention, and notification failure behavior require stakeholder confirmation.	Follow-up decisions required

6.3 Ambiguities and Missing Requirements to Resolve

Before implementation, the following details should be finalized with stakeholders:

- **Appointment policy:** Define appointment duration, booking limits, cancellation deadlines, and rescheduling rules.
- **Doctor availability:** Define how holidays, leave, and unexpected schedule changes are handled.
- **Notification failures:** Decide whether failed notifications are retried and how delivery failures are reported.

- **Data retention:** Define how long patient and consultation records must be retained and who may authorize changes or deletion.
- **Backup and recovery:** Set backup frequency, recovery time objectives, and recovery point objectives.
- **Security and privacy:** Define session timeout, password reset, account lockout, audit-log retention, and permission-review procedures.
- **Performance:** Agree on the expected number of concurrent users and the load-test environment.
- **Report access:** Define which user roles can view, export, or share each report.

6.4 Suggested Improvements

1. Conduct requirement-review meetings with representatives from each stakeholder group.
2. Assign a unique identifier to every approved requirement and maintain a requirements traceability matrix.
3. Prepare acceptance criteria for each major feature before development.
4. Perform concurrent-booking tests to verify that only valid appointments are confirmed.
5. Conduct security testing to verify that unauthorized users cannot access consultation records or administrative functions.
6. Perform usability testing with patients and hospital staff.
7. Review the SRS whenever approved changes are made to scope, policy, or functionality.
8. Obtain stakeholder approval of the final SRS before detailed design and implementation.

Conclusion

The Smart Healthcare Appointment and Patient Management System provides a structured solution to appointment scheduling and healthcare administrative challenges. This SRS defines the business context, stakeholder requirements, functional and non-functional requirements, actors, use cases, data structures, interfaces, assumptions, and constraints. The proposed UML use case diagram, system architecture, and entity relationship diagram support clear communication of the system design. Requirement traceability and validation help identify inconsistencies and missing details before development, creating a sound foundation for implementation, testing, maintenance, and future enhancement.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation (15%)	SRS is well-structured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25